

EEG-Based BCI for Robotic Hand Control: Full Report

Jonathan Davanzo

Contents

1	Introduction	3
2	System Overview	3
3	Data Acquisition	3
3.1	EEG Hardware and Sampling	3
3.2	Real-Time Streaming	3
4	Signal Preprocessing	4
4.1	Temporal Compression	4
4.2	Independent Component Analysis	4
5	Feature Extraction	4
6	Event Annotation and Labeling	4
6.1	Event-Marking Interface	4
6.2	Label Schema	4
6.2.1	Finger Labels	4
6.2.2	Action Labels	5
7	Window Extraction	5
8	Neural Network Architecture	5
8.1	Model Design	5
8.2	Training Objective	5
9	Training Procedure	6
10	Evaluation and Calibration	6
10.1	Deterministic Evaluation	6
10.2	Bayesian Uncertainty Estimation	6
11	Data and Model Validation	6
12	Real-Time Inference and Safety Gating	6
12.1	Confidence-Weighted Actuation	6
12.2	Velocity Scaling	6
13	Experiment Logging and Reproducibility	7
14	Automation and Execution	7

15 Ethical and Safety Considerations	7
16 Intended Applications	7
17 Results	7
18 Discussion	7
19 Conclusion	7
A Appendix: Reproducibility Checklist	8

Abstract

This report presents a real-time, uncertainty-aware brain–computer interface (BCI) for EEG-based finger and hand action classification with safe robotic actuation. We describe the end-to-end pipeline from data acquisition and labeling through model training, evaluation, and live deployment, and outline the safety and reproducibility mechanisms that enable reliable demonstrations.

1 Introduction

EEG-based BCIs enable hands-free interaction by decoding neural activity into actions. However, noise, artifacts, and uncertainty can degrade reliability, especially in live robotics. This project targets a lightweight, real-time pipeline that provides calibrated confidence and uncertainty estimates to gate actuation safely.

2 System Overview

This project implements a real-time, uncertainty-aware brain–computer interface (BCI) for EEG-based finger and hand action classification, designed for safe robotic actuation and interactive demonstration. The system processes raw EEG data acquired from a Muse 2 headset, performs online signal cleaning and feature extraction, trains and evaluates neural network models, and deploys real-time inference with calibrated confidence and Bayesian uncertainty estimates.

The complete pipeline is modular and consists of five primary stages:

1. EEG streaming and recording
2. Event annotation and window extraction
3. Neural network training
4. Model evaluation, calibration, and validation
5. Real-time inference with safety gating

All stages are orchestrated via an automated experiment runner while preserving reproducibility, subject anonymity, and traceable experiment metadata.

3 Data Acquisition

3.1 EEG Hardware and Sampling

EEG data are acquired using a Muse 2 EEG headset, which provides four channels: TP9, AF7, AF8, and TP10. Data are streamed at a nominal sampling rate of 256 Hz using the Lab Streaming Layer (LSL) protocol. The system automatically resolves and connects to the active EEG stream.

3.2 Real-Time Streaming

Incoming EEG samples are buffered using a sliding window approach. Each window spans 0.25 seconds (64 samples per channel). This window duration balances temporal resolution with computa-

tional latency, enabling real-time inference while preserving discriminative neural features.

4 Signal Preprocessing

4.1 Temporal Compression

To reduce redundant computation and stabilize downstream ICA fitting, temporal compression is applied by removing consecutive duplicate samples across channels. This preserves signal transitions while reducing effective sample counts.

4.2 Independent Component Analysis

Artifact removal is performed using FastICA applied to each compressed window:

1. EEG samples are standardized using a z-score transformation.
2. ICA is fitted once per session and reused thereafter.
3. Independent components are analyzed by variance.
4. Components exceeding twice the median variance are treated as artifacts and suppressed.
5. Cleaned EEG is reconstructed via inverse ICA transformation.

This adaptive approach enables ocular and muscular artifact suppression without predefined templates.

5 Feature Extraction

From each cleaned EEG window, a full time-series segment is retained ($64 \text{ timesteps} \times 4 \text{ channels}$). A simple channel-wise mean feature vector is also computed for diagnostics:

$$\mathbf{x} = [\mu_{TP9}, \mu_{AF7}, \mu_{AF8}, \mu_{TP10}]$$

These features are computationally lightweight, suitable for real-time inference, and consistent across training and evaluation.

6 Event Annotation and Labeling

6.1 Event-Marking Interface

A dedicated event-marking user interface allows post-hoc labeling of EEG data. The interface supports click-and-drag duration marking, keyboard-based labeling, undo/redo functionality, autosave, and export to EEGLAB-compatible `.set` files.

6.2 Label Schema

To align with BCI competition standards and multi-task learning, labels are structured hierarchically.

6.2.1 Finger Labels

- Thumb

- Index
- Middle
- Ring
- Pinky
- Rest (optional)

6.2.2 Action Labels

- Open
- Close
- Rest / No movement

Each EEG window is assigned both a finger identity and an action label, enabling multi-head neural network training.

7 Window Extraction

Annotated EEG recordings are segmented into fixed-length labeled windows and stored in `eeg_windows.npz` with shape $[N, T, C]$. A compact summary is saved to `eeg_windows.csv`. Windows inherit labels from overlapping annotated events. Ambiguous windows are either resolved via majority labeling or excluded.

This windowed dataset serves as the sole input for model training, evaluation, and validation.

8 Neural Network Architecture

8.1 Model Design

The primary model, `CNNLSTMFingerActionNet`, is a multi-head neural network comprising:

- A shared feature encoder
- A finger classification head
- An action classification head

Dropout layers are included throughout the network to support Bayesian uncertainty estimation via Monte Carlo dropout.

8.2 Training Objective

The model is trained using a joint loss:

$$\mathcal{L} = \mathcal{L}_{\text{finger}} + \mathcal{L}_{\text{action}}$$

where each term is a categorical cross-entropy loss.

9 Training Procedure

Data are split into training (80%) and test (20%) sets using subject-wise splits when available; otherwise stratified sampling is used. Feature normalization is fitted exclusively on the training set to prevent data leakage. The model is trained using the Adam optimizer for a fixed number of epochs with dropout enabled.

All artifacts, including trained weights, scalers, and predictions, are saved to disk.

10 Evaluation and Calibration

10.1 Deterministic Evaluation

For standard metrics, dropout is disabled. Accuracy, confusion matrices, and Expected Calibration Error (ECE) are computed on the held-out test set.

10.2 Bayesian Uncertainty Estimation

Uncertainty is estimated using Monte Carlo dropout:

1. Dropout remains active during inference.
2. Multiple stochastic forward passes are performed.
3. Predictive mean and standard deviation are computed.

Confidence is defined as the maximum predictive mean probability, while uncertainty is defined as the mean predictive standard deviation.

11 Data and Model Validation

The Deepchecks tabular validation framework is used to detect data leakage, distribution shifts, label inconsistencies, and anomalous model behavior. Validation operates on the same windowed dataset used for training.

12 Real-Time Inference and Safety Gating

12.1 Confidence-Weighted Actuation

Live actuation is gated by an adaptive confidence threshold that increases with uncertainty. Actuation is permitted only when predictions are stable across consecutive frames.

12.2 Velocity Scaling

Motor velocity is scaled according to:

$$v = \text{confidence} \times (1 - \text{uncertainty})$$

ensuring conservative behavior under uncertainty.

13 Experiment Logging and Reproducibility

Experiments are logged using anonymous subject identifiers, deterministic experiment hashes, and step-wise metadata records. This enables full traceability while preserving subject privacy.

14 Automation and Execution

All pipeline stages are orchestrated using an automated experiment runner that verifies dependencies, executes steps in order, and generates evaluation reports and figures.

15 Ethical and Safety Considerations

Subject anonymity is strictly preserved. Robotic actuation defaults to a disabled state under uncertainty, and conservative thresholds are enforced during live demonstrations.

16 Intended Applications

The system is designed for science fairs, research demonstrations, grant submissions, educational BCI prototypes, and safe human–robot interaction research.

17 Results

This section summarizes model accuracy, calibration, and live performance metrics. Replace the placeholders below with actual results once finalized.

- Finger classification accuracy: *TBD*
- Action classification accuracy: *TBD*
- Calibration (ECE): *TBD*
- Live inference latency: *TBD*

18 Discussion

The modular pipeline supports rapid iteration and safe demonstrations, but performance depends on data quality, labeling fidelity, and subject variability. Future work includes richer features, subject-adaptive calibration, and additional uncertainty-aware gating strategies.

19 Conclusion

This report documents a complete EEG-to-robotics pipeline that prioritizes real-time performance, calibrated uncertainty, and reproducible experimentation.

Acknowledgments

Thanks to mentors, collaborators, and participants who supported data collection and feedback.

References

- [1] Muse 2 Headset, InteraXon, Inc. Product documentation and specifications.
- [2] Lab Streaming Layer (LSL). <https://labstreaminglayer.readthedocs.io/>
- [3] Deepchecks: A validation framework for tabular data and models.

A Appendix: Reproducibility Checklist

- Fixed train/test split with stratification
- Normalization fit on training set only
- Saved scalers, weights, and predictions
- Logged experiment metadata and hashes